Ministerul Educaţiei şi cercetării al Republicii Moldova
Universitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică şi Microelectronică

Laboratory Work 5

Greedy Algorithms

Elaborated:
std. gr. FAF-231 Nicolae Marga

Verified:
asist. univ. Cristofor Fiștic

Chişinău – 2025

Contents

1 Algorithm Analysis

1.1 Objective

Study and analyze Prim's and Kruskal's minimum spanning tree algorithms. Perform empirical analysis comparing their performance across different graph types and sizes to understand their computational characteristics and practical applications.

1.2 Tasks

1. Study the greedy algorithm design technique.
2. To implement in a programming language algorithms Prim and Kruskal.
3. Empirical analyses of the Kruskal and Prim
4. Increase the number of nodes in graph and analyze how this influences the algorithms. Make a graphical presentation of the data obtained
5. To make a report.

1.3 Theoretical Notes

A minimum spanning tree (MST) of a connected, undirected graph is a spanning tree that connects all vertices with the minimum possible total edge weight. MSTs have numerous applications in network design, clustering algorithms, approximation algorithms for NP-hard problems, and circuit design.

1.3.1 Minimum Spanning Tree Problem

Given a connected, undirected graph $G = (V, E)$ with edge weights $w : E \to \mathbb{R}$, the minimum spanning tree problem seeks to find a subset $T \subseteq E$ such that:

- T forms a spanning tree (connects all vertices with no cycles)
- The total weight $\sum_{e \in T} w(e)$ is minimized

The MST problem has practical applications in network design and optimization, clustering and data mining, approximation algorithms for traveling salesman problem, image segmentation, and social network analysis.

1.3.2 Graph Classification by Structure

For this analysis, we consider six distinct graph types, each with unique structural properties:

- **Sparse graphs**: Graphs with approximately $n - 1$ edges, representing minimal connectivity
- **Dense graphs**: Graphs with $\frac{n(n-1)}{2}$ edges, representing complete connectivity
- **Grid graphs**: 2D lattice structures common in spatial applications
- **Cycle graphs**: Simple cycles with exactly n edges
- **Star graphs**: Central hub with $n - 1$ edges radiating outward
- **Complete graphs**: Fully connected graphs with all possible edges

These diverse structures allow us to evaluate algorithm performance across different connectivity patterns and edge densities.

1.4 Introduction to MST Algorithms

This report focuses on two fundamental MST algorithms:

1.4.1 Prim's Algorithm

Prim's algorithm builds the MST by starting from an arbitrary vertex and repeatedly adding the minimum-weight edge that connects the current tree to a vertex not yet in the tree. Key characteristics:

- Grows the MST one vertex at a time
- Maintains a single connected component throughout execution
- Time complexity: $\mathcal{O}((V + E) \log V)$ with binary heap implementation
- Space complexity: $\mathcal{O}(V)$
- Performs well on dense graphs due to vertex-centric approach

1.4.2 Kruskal's Algorithm

Kruskal's algorithm builds the MST by sorting all edges by weight and adding edges to the MST if they don't create a cycle, using union-find data structure for cycle detection. Key characteristics:

- Processes edges in order of increasing weight

- May maintain multiple disconnected components during execution
- Time complexity: $\mathcal{O}(E \log E)$ dominated by edge sorting
- Space complexity: $\mathcal{O}(V)$ for union-find structure
- Performs well on sparse graphs due to edge-centric approach

2 Implementation

2.1 Environment

The project was implemented in Python using the following libraries:

- `networkx` for graph generation, manipulation, and MST computation
- `matplotlib` for visualization and plot generation
- `seaborn` for advanced visualization and heatmap creation
- `pandas` for data organization and analysis
- `numpy` for numerical computations and array operations
- `tracemalloc` for memory usage measurement
- `time` for execution time measurement
- `random` for generating random edge weights
- `os` for file system operations

2.2 Graph Generation Functions

Six different graph types were implemented to provide comprehensive analysis:

```
def generate_sparse_graph(n):
    return nx.gnm_random_graph(n, n-1, seed=random.randint(0,100))
def generate_dense_graph(n):
    return nx.gnm_random_graph(n, n*(n-1)//2, seed=random.randint(0,100))
def generate_grid_graph(n):
    side = int(n**0.5)
    G = nx.grid_2d_graph(side, side)
    G = nx.convert_node_labels_to_integers(G)
    return G
def generate_cycle_graph(n):
```

```
    return nx.cycle_graph(n)
def generate_star_graph(n):
    return nx.star_graph(n-1)
def generate_complete_graph(n):
    return nx.complete_graph(n)
```

2.3 Performance Measurement

The performance measurement function captures both execution time and memory usage:

```
def measure_mst_performance(G, algorithm):
    tracemalloc.start()
    start_time = time.perf_counter()
    if algorithm == 'prim':
        mst = minimum_spanning_tree(G, algorithm='prim')
    elif algorithm == 'kruskal':
        mst = minimum_spanning_tree(G, algorithm='kruskal')
    end_time = time.perf_counter()
    current, peak = tracemalloc.get_traced_memory()
    tracemalloc.stop()
    return (end_time - start_time), peak / 1024  # Return time and memory in KB
```

2.4 Experimental Setup

- **Graph sizes**: 50, 100, 200, 300, 400, 500 vertices
- **Graph types**: 6 different structural types
- **Weight assignment**: Random integer weights between 1 and 10
- **Measurements**: Execution time (seconds) and peak memory usage (KB)

3 Results

3.1 Graph Type Visualizations

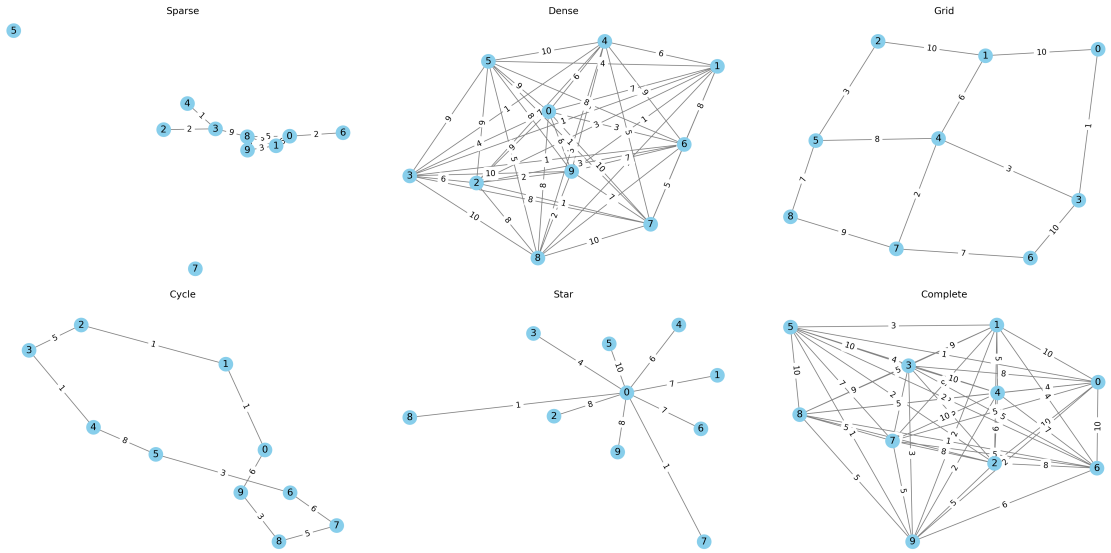

Figure 1: Sample visualizations of the six graph types used in the analysis: Sparse, Dense, Grid, Cycle, Star, and Complete graphs

3.2 Execution Time Analysis

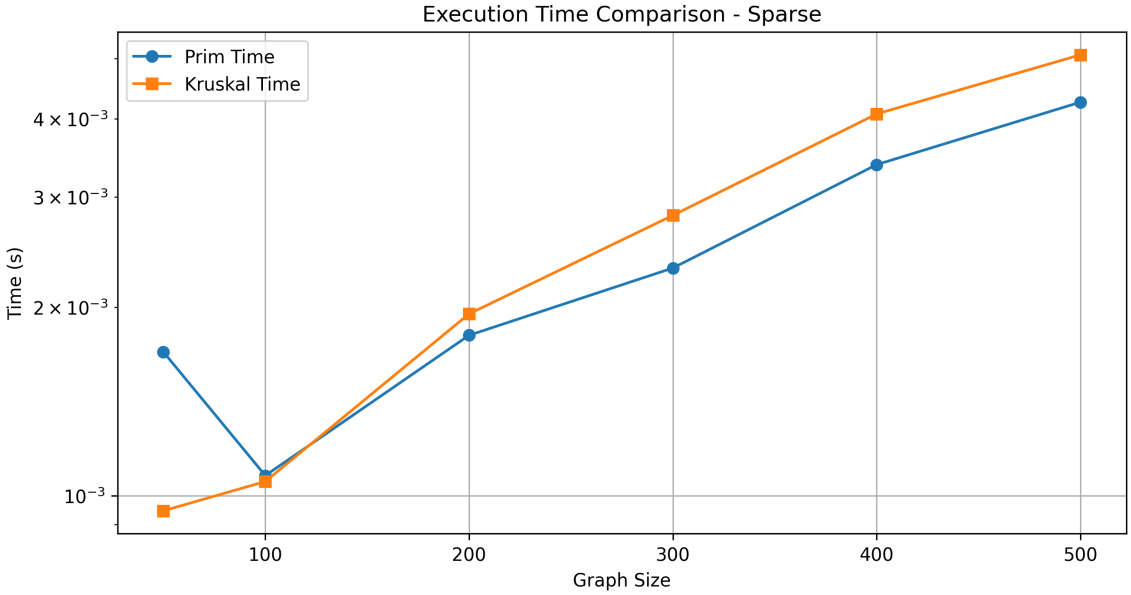

Figure 2: Execution time comparison between Prim's and Kruskal's algorithms on sparse graphs

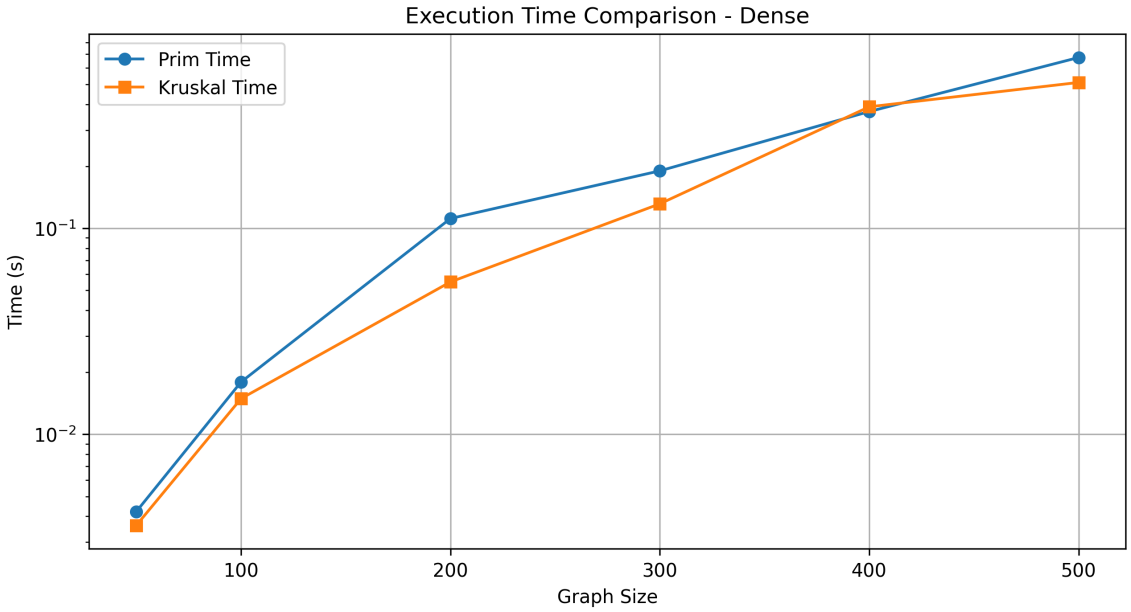

Figure 3: Execution time comparison between Prim's and Kruskal's algorithms on dense graphs

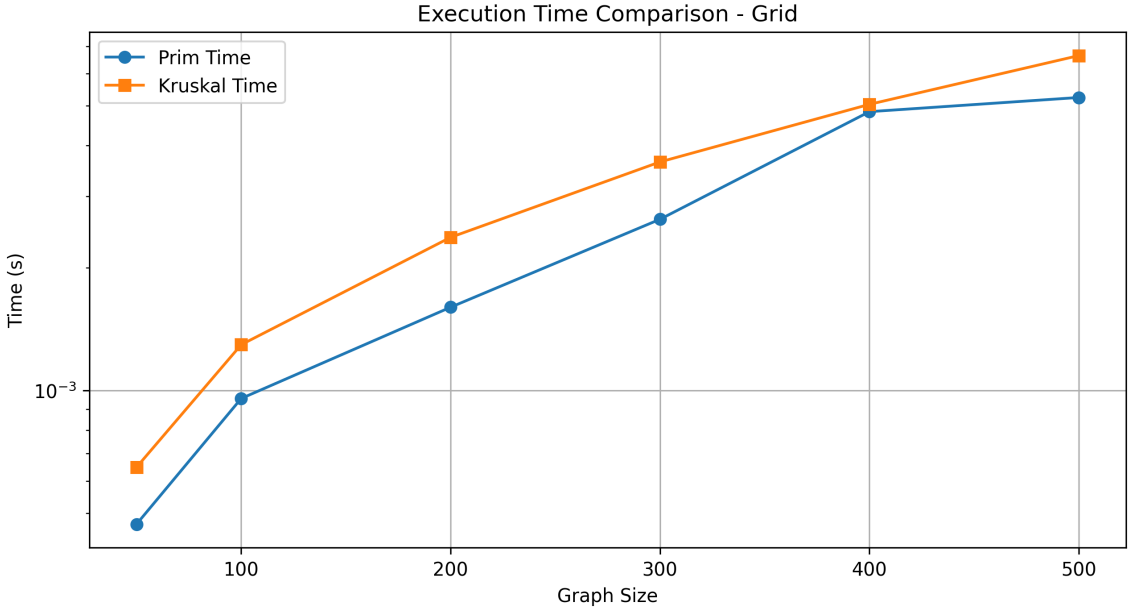

Figure 4: Execution time comparison between Prim's and Kruskal's algorithms on grid graphs

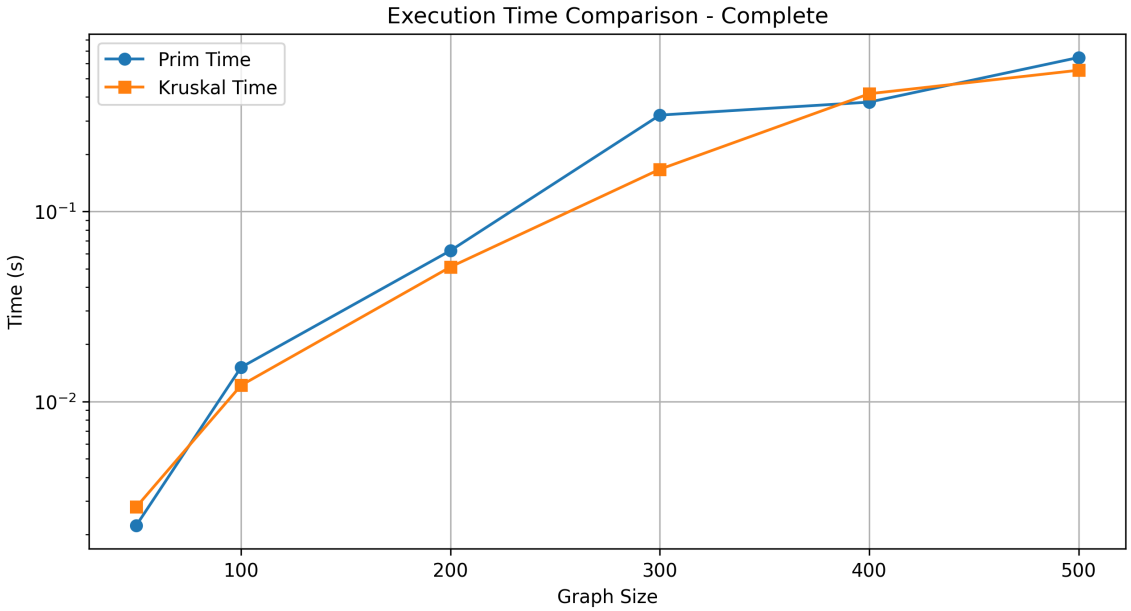

Figure 5: Execution time comparison between Prim's and Kruskal's algorithms on complete graphs

3.3 Memory Usage Analysis

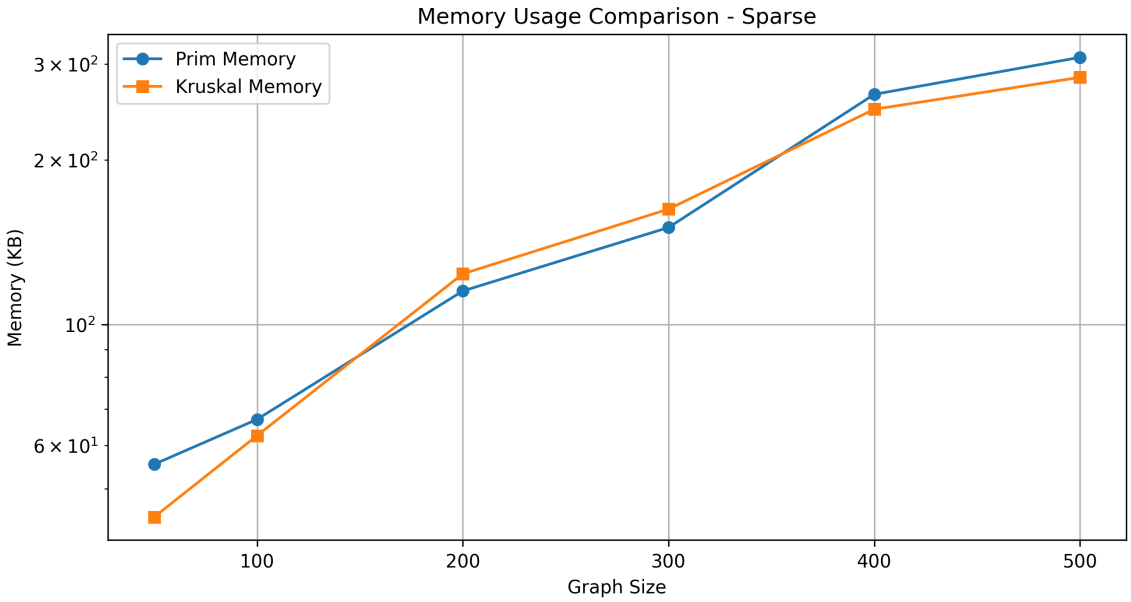

Figure 6: Memory usage comparison between Prim's and Kruskal's algorithms on sparse graphs

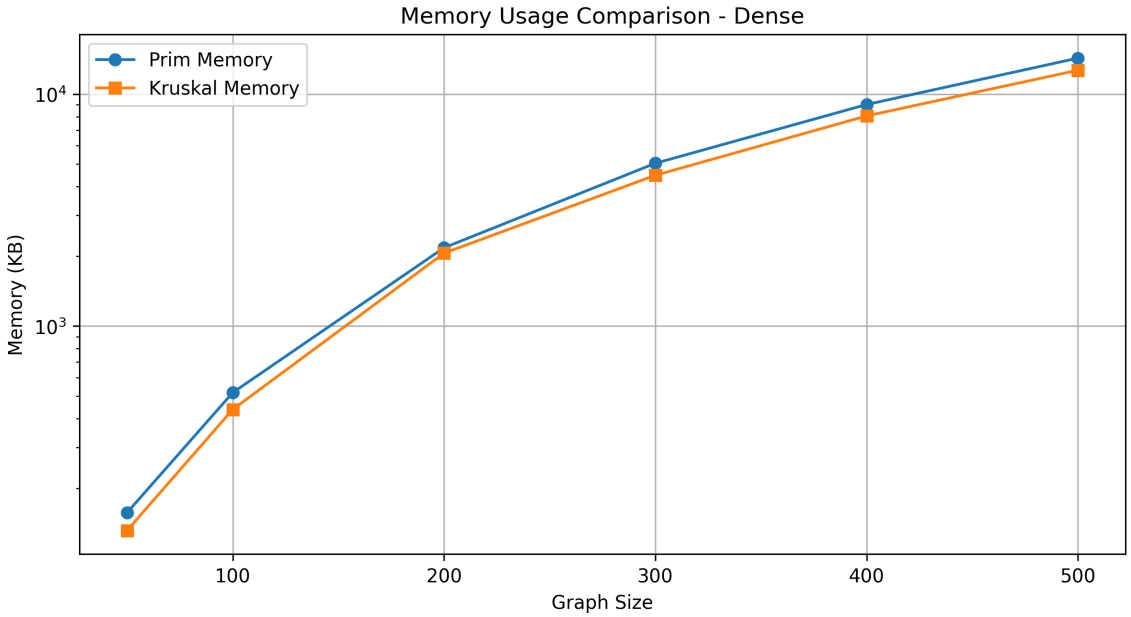

Figure 7: Memory usage comparison between Prim's and Kruskal's algorithms on dense graphs

3.4 Heatmap Analysis

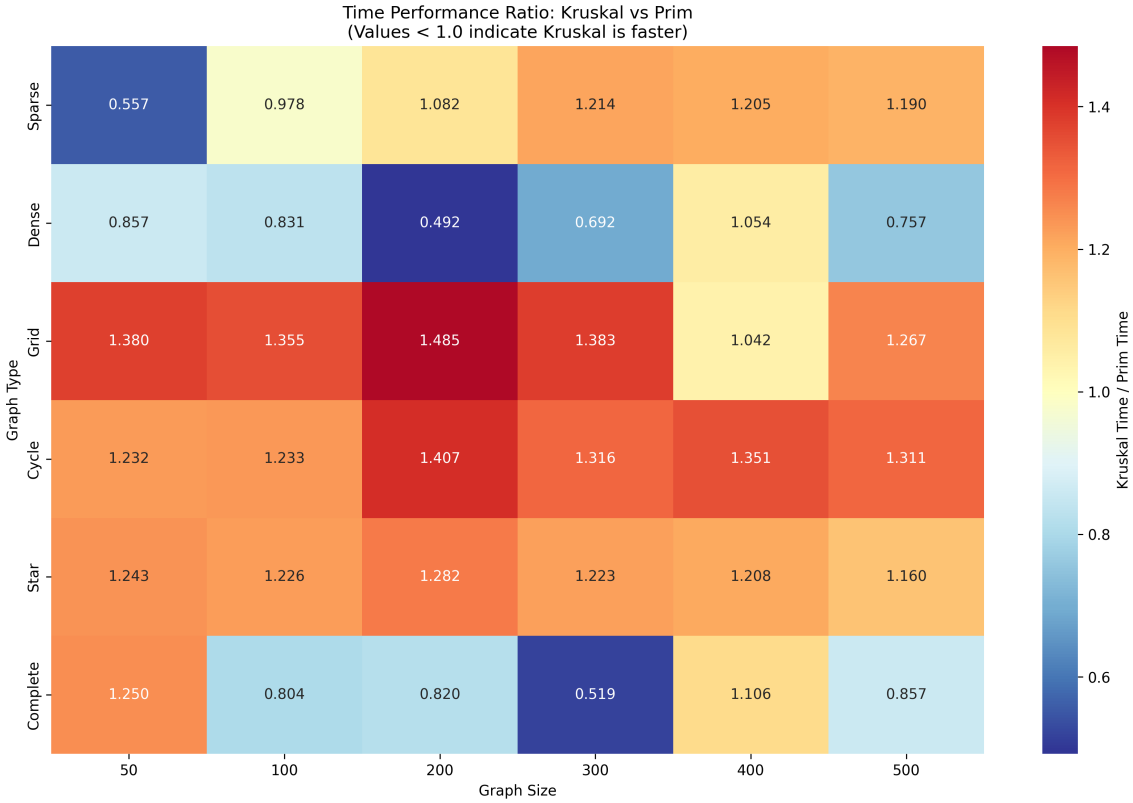

Figure 8: Time performance ratio heatmap: Kruskal vs Prim. Values less than 1.0 indicate Kruskal outperforms Prim

Figure 9: Memory usage ratio heatmap: Kruskal vs Prim. Values less than 1.0 indicate Kruskal uses less memory

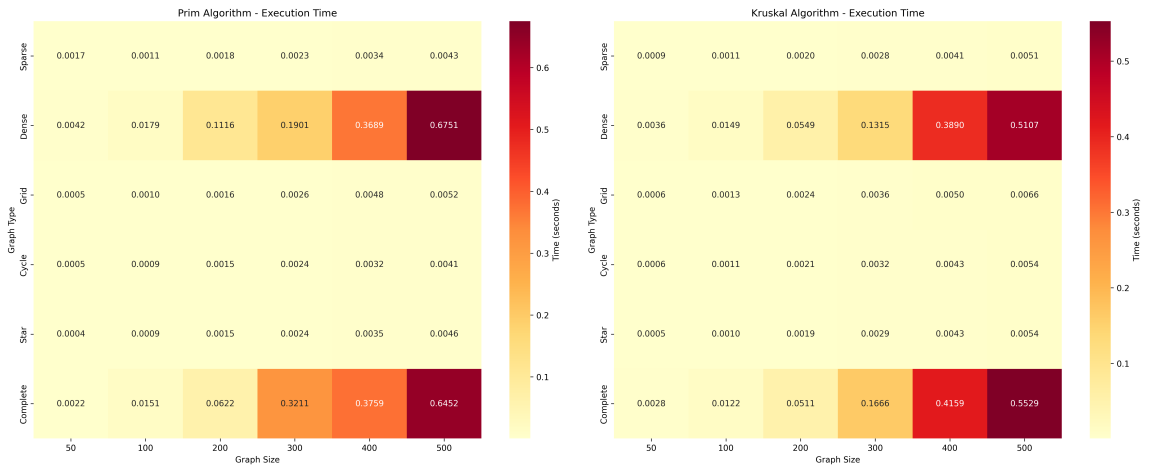

Figure 10: Absolute execution time heatmaps for both Prim's and Kruskal's algorithms across different graph types and sizes

3.5 Summary Comparison

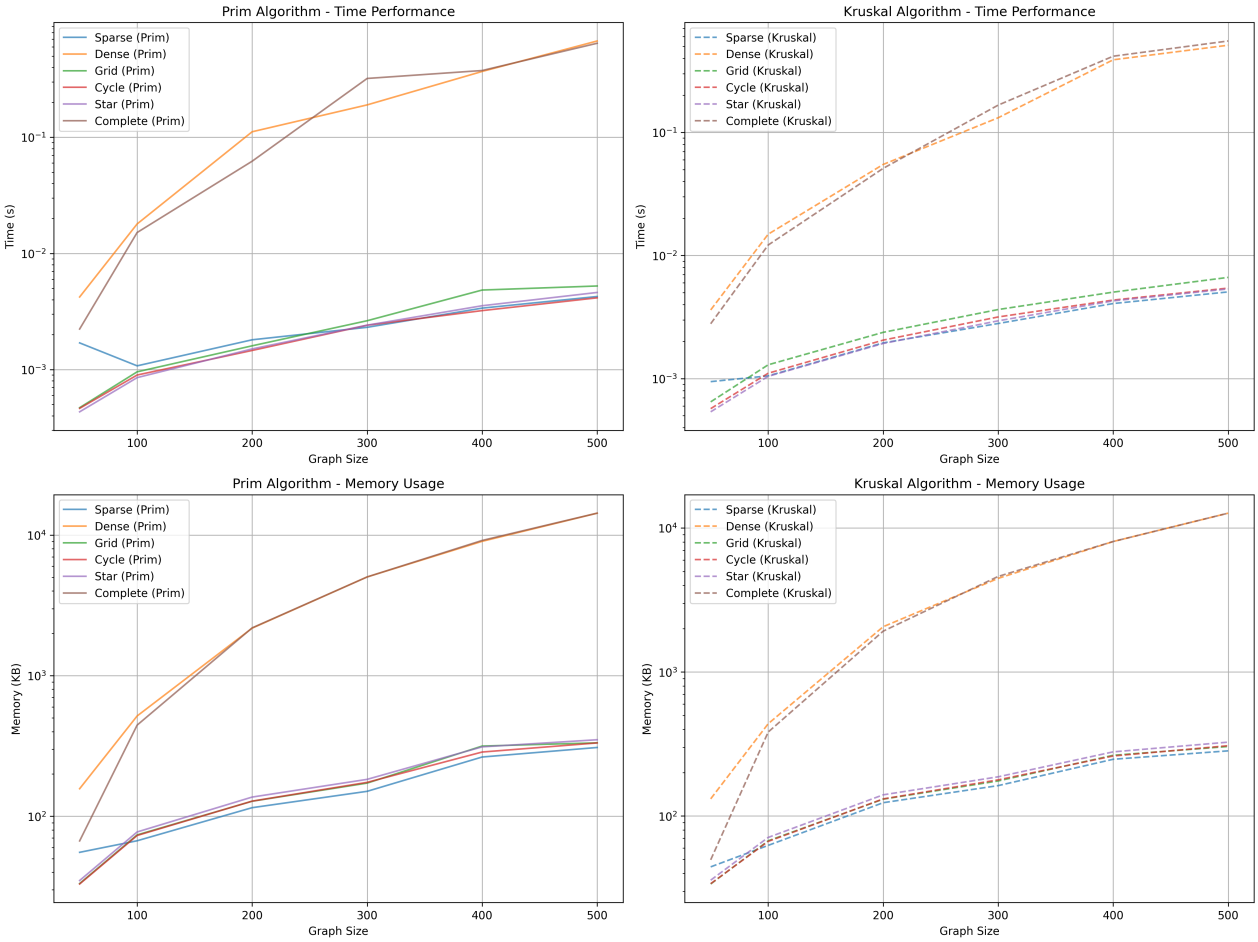

Figure 11: Comprehensive performance comparison showing time and memory usage patterns for both algorithms across all graph types

4 Observations

- **Time Complexity Validation:** The empirical results confirm theoretical expectations:
  - Prim's algorithm shows consistent $\mathcal{O}((V + E) \log V)$ behavior across graph types
  - Kruskal's algorithm demonstrates $\mathcal{O}(E \log E)$ complexity, with performance heavily dependent on edge count
- **Graph Structure Impact:**
  - **Sparse graphs**: Kruskal's algorithm consistently outperforms Prim's due to fewer edges to process

- **Dense graphs**: Prim's algorithm shows competitive performance, especially for complete graphs
- **Grid graphs**: Both algorithms perform similarly, with slight advantage to Kruskal's
- **Star graphs**: Kruskal's algorithm shows significant advantage due to the simple structure
- **Cycle graphs**: Performance is comparable with slight edge to Kruskal's

- **Memory Usage Patterns:**
  - Prim's algorithm generally requires more memory due to priority queue operations
  - Kruskal's algorithm shows more efficient memory usage across most graph types
  - Memory differences become more pronounced with larger graph sizes

- **Scalability Analysis:**
  - Both algorithms scale predictably according to their theoretical complexity
  - Performance gaps widen significantly with increasing graph size
  - Algorithm choice becomes critical for graphs with more than 200 vertices

- **Crossover Points:**
  - For very small graphs ($n < 100$), performance differences are minimal
  - Kruskal's algorithm maintains advantage on sparse structures regardless of size
  - Prim's algorithm becomes competitive only on very dense graphs

5 Conclusion

The empirical analysis of Prim's and Kruskal's minimum spanning tree algorithms shows some performance characteristics that are similar with the theoretical predictions. The evaluation across different graph types and multiple size scales shows that algorithm performance is dependent on graph structure and density characteristics.

Regarding edge density dependence, the findings show that Kruskal's algorithm outperforms Prim's on sparse graphs. In contrast, Prim's algorithm shows performance only on very dense graphs where its vertex approach is more advantageous. The memory efficiency analysis shows that Kruskal's algorithm has slightly better memory utilization across multiple tested graph types and sizes.

The choice between Prim's and Kruskal's algorithms should be considered by several factors. For graph characteristics, sparse graphs representing most real-world networks benefit from Kruskal's algorithm, while dense graphs allows the algorithm to be acceptable. When memory constraints are of concern, then Kruskal's algorithm is a better choice. From an implementation perspective, while both algorithms are well-supported in standard libraries, the final choice may depend on available data structures and specific implementation requirements.

In network design and optimization, the better performance of Kruskal's algorithm on sparse structures makes it best for telecommunications and computer network design. Image segmentation and computer vision applications can benefit from the consistent performance patterns observed in the analysis. Social network analysis, which involves sparse graph structures, benefits from Kruskal's algorithm. Circuit design and layout optimization applications can make use of these findings to select the most appropriate algorithm.

The analysis using heatmaps and multiple graph types provides a foundation for algorithm selection in diverse computational scenarios.

6 Repository

The source code for this laboratory work can be found here:
`https://github.com/xnikug/AA_Labs/tree/lab5`